

***BRAC*'s**

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 3 Practical No: 5

Roll no: 66 PRN No: 12411609 Name of Student: Laukika Shinde

Subject Name: Deep Learning

Problem Statement: Implement and compare RNN, LSTM, and GRU models for sequence classification, and analyze their performance using appropriate evaluation metrics.

Algorithm:

- 1) Import the required libraries.
- 2) Load the IMDB movie review dataset.
- 3) Preprocess and pad the sequences to a fixed length.
- 4) Split the dataset into training and testing sets.
- 5) Build and compile the RNN model.
- 6) Build and compile the LSTM model.
- 7) Build and compile the GRU model.
- 8) Train all three models using the training data.
- 9) Generate predictions using the test data.
- 10) Calculate accuracy, precision, recall and F1-score.
- 11) Compare the performance of RNN, LSTM and GRU models using a graph.

Code:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from tensorflow.keras.datasets import imdb

from tensorflow.keras.preprocessing.sequence import pad_sequences

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Embedding, SimpleRNN, LSTM, GRU, Dense
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

```
num_words = 10000
```

```
max_length = 200
```

```
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=num_words)
```

```
X_train = pad_sequences(X_train, maxlen=max_length, padding='post', truncating='post')
```

```
X_test = pad_sequences(X_test, maxlen=max_length, padding='post', truncating='post')
```

```
def create_rnn():
```

```
    model = Sequential([
```

```
        Embedding(num_words, 64),
```

```
        SimpleRNN(64),
```

```
        Dense(1, activation='sigmoid')
```

```
    ])
```

```
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

```
    return model
```

```
def create_lstm():
```

```
    model = Sequential([
```

```
        Embedding(num_words, 64),
```

```
        LSTM(64),
```

```
        Dense(1, activation='sigmoid')
```

```

    ])

    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

    return model


def create_gru():

    model = Sequential([

        Embedding(num_words, 64),

        GRU(64),

        Dense(1, activation='sigmoid')

    ])

    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

    return model


rnn_model = create_rnn()

lstm_model = create_lstm()

gru_model = create_gru()


rnn_model.fit(X_train, y_train, epochs=5, batch_size=128, validation_split=0.2)

lstm_model.fit(X_train, y_train, epochs=5, batch_size=128, validation_split=0.2)

gru_model.fit(X_train, y_train, epochs=5, batch_size=128, validation_split=0.2)


models = {

    "RNN": rnn_model,

    "LSTM": lstm_model,

```

```

    "GRU": gru_model
}

results = []

for name, model in models.items():

    probabilities = model.predict(X_test)

    predictions = (probabilities >= 0.5).astype(int).flatten()

    accuracy = accuracy_score(y_test, predictions)

    precision = precision_score(y_test, predictions)

    recall = recall_score(y_test, predictions)

    f1 = f1_score(y_test, predictions)

    results.append([name, accuracy, precision, recall, f1])

results_df = pd.DataFrame(results, columns=["Model", "Accuracy", "Precision", "Recall", "F1-Score"])

print(results_df)

results_df.set_index("Model").plot(kind="bar", figsize=(10, 6))

plt.title("RNN vs LSTM vs GRU Performance")

plt.ylabel("Score")

plt.ylim(0, 1)

plt.xticks(rotation=0)

plt.show()

```

Output:

```
Epoch 1/5
157/157 ————— 22s 106ms/step - accuracy: 0.5033 - loss: 0.6954 - val_accuracy: 0.5088 - val_loss: 0.6927
Epoch 2/5
157/157 ————— 19s 121ms/step - accuracy: 0.5950 - loss: 0.6538 - val_accuracy: 0.5074 - val_loss: 0.7048
Epoch 3/5
157/157 ————— 16s 103ms/step - accuracy: 0.6837 - loss: 0.5229 - val_accuracy: 0.5038 - val_loss: 0.8046
Epoch 4/5
157/157 ————— 17s 106ms/step - accuracy: 0.7424 - loss: 0.4031 - val_accuracy: 0.5126 - val_loss: 0.9269
Epoch 5/5
157/157 ————— 17s 111ms/step - accuracy: 0.7621 - loss: 0.3622 - val_accuracy: 0.5050 - val_loss: 1.0001
Epoch 1/5
157/157 ————— 59s 361ms/step - accuracy: 0.5192 - loss: 0.6924 - val_accuracy: 0.5206 - val_loss: 0.6929
Epoch 2/5
157/157 ————— 82s 360ms/step - accuracy: 0.5800 - loss: 0.6760 - val_accuracy: 0.5062 - val_loss: 0.6957
Epoch 3/5
157/157 ————— 55s 352ms/step - accuracy: 0.5597 - loss: 0.6825 - val_accuracy: 0.5476 - val_loss: 0.6848
Epoch 4/5
157/157 ————— 55s 352ms/step - accuracy: 0.5581 - loss: 0.6667 - val_accuracy: 0.5358 - val_loss: 0.6852
Epoch 5/5
157/157 ————— 63s 400ms/step - accuracy: 0.5985 - loss: 0.6472 - val_accuracy: 0.5676 - val_loss: 0.6659

Epoch 1/5
157/157 ————— 68s 415ms/step - accuracy: 0.5141 - loss: 0.6926 - val_accuracy: 0.5248 - val_loss: 0.6909
Epoch 2/5
157/157 ————— 63s 402ms/step - accuracy: 0.6188 - loss: 0.6468 - val_accuracy: 0.5934 - val_loss: 0.6637
Epoch 3/5
157/157 ————— 65s 415ms/step - accuracy: 0.7027 - loss: 0.5863 - val_accuracy: 0.6628 - val_loss: 0.6149
Epoch 4/5
157/157 ————— 65s 414ms/step - accuracy: 0.6744 - loss: 0.5994 - val_accuracy: 0.5358 - val_loss: 0.6728
Epoch 5/5
157/157 ————— 82s 414ms/step - accuracy: 0.8186 - loss: 0.4264 - val_accuracy: 0.8388 - val_loss: 0.3770
782/782 ————— 11s 14ms/step
782/782 ————— 24s 31ms/step
782/782 ————— 22s 28ms/step
Model Accuracy Precision Recall F1-Score
0 RNN 0.50056 0.500591 0.47400 0.486933
1 LSTM 0.55448 0.541764 0.70672 0.613344
2 GRU 0.82896 0.802531 0.87264 0.836118
```

